COMP3270 Assignment 2: Playing Pacman

Release date (course schedule): 9 October 2026
Submission deadline: 8 November 2026
Assignment weight: 10% of the course
Grading TA: Junshi

Marks and Assessment

This is an **individual assignment**, marked out of **100**. Q0 is an ungraded warm-up. You will build game rules, improve a one-step player, then search ahead using minimax and expectimax. Reuse the functions you complete along the way.

Question	Code	Written answers	Total
Q0: Manhattan distance	0	0	0
Q1: Game rules and random play	20	0	20
Q2: Reflex Pacman, one ghost	10	5	15
Q3: Multiple agents and turn depth	3	2	5
Q4: Reflex Pacman, multiple ghosts	7	3	10
Q5: Minimax	20	5	25
Q6: Expectimax	16	9	25
Total	**76**	**24**	**100**

Code marks assess correctness, the stated algorithm and function contracts, including edge cases. Written marks assess accurate measurements and explanations. Public test counts are not mark allocations. Passing them does not establish full correctness. Q2/Q4 also require a reasoned evaluation function: a particular win percentage is **not** required. Later questions will be assessed independently where possible, using correct earlier helpers, so one early error need not lose all later marks. Written answers are marked separately.

Q2 code marks cover contracts (2), meaningful food/ghost features (4), and weights/move ranking (4). Q4 code marks cover both evaluation contracts (2), food and all-ghost features (3), and the added feature with a controlled removal comparison (2). These formulas are open: there is no required hidden formula. Q1 allocates 6 marks to legal actions, 12 to state transitions and 2 to supplied RNG use. Q6 reuses the search structure from Q5; its 16 code and 9 written marks place more emphasis on reasoning about chance outcomes.

Academic Integrity: No AI

Do not use generative-AI tools for any part of this assignment. This includes interpreting questions, planning solutions, generating or completing code, debugging, and drafting or revising written answers. The rule applies to chatbots, coding agents and AI features built into editors. Disable AI assistance while working on A2.

Complete the work independently. Do not share code or written answers, copy another person's solutions, or offer your work for copying. You may use the supplied course materials and ordinary, non-AI documentation; acknowledge sources you use. Standard non-AI editor features are allowed.

Ask the teaching team or use Ed when you need clarification. Do not post your solution code publicly. You must understand your submission and may be asked to explain your code and reasoning.

Getting Started

You need **Python 3.10 or later**, a plain-text code editor and a terminal. Only Python's standard library is used; no additional packages are required.

1. Extract and Open the Student Folder

Download `a_02.zip`, extract it, and open `a_02/student/`. Keep `q0.py`-`q6.py`, `play.py`, `maps/`, `support/`, `test_cases/` and the five answer templates in this folder. Work in the extracted folder, not the ZIP preview.

On **Windows**, open `student` in File Explorer, type `powershell` in its address bar and press Enter. On **macOS**, open Terminal, type `cd` (including the space), drag the `student` folder from Finder into the terminal, then press Enter.

Check	Windows PowerShell	macOS Terminal
Python version	`python --version`	`python3 --version`
Correct folder	`dir q0.py`	`ls q0.py`

The version must be 3.10 or later and the second command must find `q0.py`. If Python is unavailable, ask the teaching team for setup help. **The examples below use `python3`; on Windows use `python` instead.** Type the command only, without the `$` shown in transcripts.

2. Edit, Save, Run and Check

Open `q0.py` in your editor. Read the function's comments, then replace its `raise NotImplementedError(...)` with your implementation. Keep four-space indentation and leave the supplied runner at the bottom unchanged. Save with Ctrl+S (Windows) or Command+S (macOS). Keep the filename `q0.py`, not `q0.py.txt`.

Run these commands from `student/`:

```
python3 q0.py
python3 q0.py --test
```

The first runs one example; the second checks **all** Q0 public cases. Before implementation, a `NotImplementedError` message is expected. Each unfinished check reports `FAILED` with the student filename, function and line. Other checks still run, so you can see progress while completing functions. After editing, save and rerun. If a test fails, inspect its first error or Expected/Actual difference. Continue with Q1 once Q0 passes **3/3**.

3. Choose How to Run a Case

All question scripts support `--list` (`-l`), `--show-maze` (`-m`), `--test` (`-t`) and `--help` (`-h`). Q1-Q6 also support `--replay` (`-r`), `--trials`, `--seed` and `--max-rounds`. All questions offer `--inspect`; Q5/Q6 also offer `--depth` and `--inspect --branches`. Q4 offers `--inspect --ablate` for feature comparisons.

A case in `test_cases/qN/` names a shared map in `maps/` and supplies inputs. Public-test expected answers are separate `_expected.json` files. Cases named `written_*` are for your written comparisons; their results are not supplied and `--test` skips them. Edit your question files and answer templates; leave `maps/`, `support/` and `test_cases/` unchanged. You do **not** need to read or edit `support/` to complete the work. For Q2, `race`, `race.json` and `test_cases/q2/race.json` name the same case. Each script accepts its own question's cases; use `--list` to find them. **A named case with `--test` checks only that case; omitting the case checks all public cases.** `--list` shows which modes each case supports.

Option	What it does
`--show-maze`	Displays the case's board without calling your functions.
`--inspect`	Runs the case's specified operation and prints your result.
`--test`	Checks your results against the public tests.
No mode option (Q1-Q6)	Runs a game from the map's initial state.
`--replay` (Q1-Q6)	Runs a game, then lets you step through its recorded moves.

For example, `east_food --inspect` checks one East move, whereas `east_food` alone runs a random game on that case's map. Q1's `random_game --inspect` runs the whole seeded game because that is the operation this case specifies. Try:

```
python3 q1.py corridor --test
python3 q1.py east_food --inspect
```

Read the Board and Game Rules

Preview the initial board before implementing any functions (complete output):

```
$ python3 q1.py corridor --show-maze
Running Q1: corridor
Map: maps/corridor.json
  0123
0 ████
1 █ W█
2 █  █
3 █  █
4 █ ·█
5 █●·█
6 ████
Score: 0 | Food remaining: 2
```

As in A1, every character is one cell: `█` wall, `●` Pacman, `·` food, one space an empty cell, and `W`, `X`, `Y`, `Z` ghosts. Coordinates are `(row, column)`, starting at zero in the upper-left. Rows increase downwards; columns increase rightwards. Column numbers with two digits are stacked above their cells. The filename identifies the map. These are moving games: walls stay fixed, players move and food disappears when eaten. A ghost covering food hides its dot; the food remains in the state. On capture, the ghost is displayed at Pacman's position.

Moves, Turns and Scoring

1. Direction labels are `'North'`, `'East'`, `'South'`, `'West'`, as in A1, in that order. They change position by `(-1, 0)`, `(0, 1)`, `(1, 0)`, `(0, -1)`. No diagonal moves. Check bounds before indexing; an open border cell is legal.
2. Neither player can enter a wall or leave the grid. A ghost cannot enter another ghost's cell. Pacman can enter a ghost's cell, and a ghost can enter Pacman's cell: both moves are legal but cause capture. Ghosts can enter food cells.
3. A turn with no legal directions is a **forced pass**, represented by `None`. Passing is not an option when a direction is legal. Invalid moves or unnecessary passes must raise `ValueError` in your successor function.
4. Each Pacman turn, including a forced pass, costs **1 point**. A ghost turn costs zero. After moving or passing, check capture **first**. Capture subtracts **500** and ends the game with winner `'Ghost'`.
5. If Pacman survives, entering a food cell removes that food and adds **10**. Eating the last food adds a further **500** and ends with winner `'Pacman'`. A collision on food loses; the food stays and no food/win bonus is awarded. Ghosts never remove food. End the game immediately, without later agents moving.
6. Start at score zero. The supplied initial-state helper sets an empty-food map to winner `'Pacman'` immediately, still at score zero. Terminal states have no actions; applying a successor returns a fresh equal state without changing score.

A full round visits Pacman and each present ghost once, in alphabetical ghost order. A stuck agent still takes its passing turn. The runner stops after the case's maximum rounds and reports **Unfinished** if nobody won. This limit is not a game win, loss or search terminal state.

The Python Interface

All assessed functions receive a supplied `problem` dictionary. You may read:

Field	Type and meaning
`grid`	Nonempty list of equally long strings; the original map layout.
`walls`	`frozenset` of wall coordinate tuples.
`start`	Pacman's initial `(row, column)` tuple.
`ghosts`	Dictionary of present ghost labels to initial coordinate tuples.
`food`	`frozenset` of initial food coordinate tuples.
`map_file`	Display filename, e.g. `maps/corridor.json`.

A **state** is a dictionary with exactly these fields:

Field	Type and meaning
`pos`	Pacman's current coordinate tuple.
`ghosts`	Dictionary of present ghost labels to current coordinate tuples.
`food`	`frozenset` of remaining food coordinate tuples.
`score`	Integer game score.
`winner`	`None` while playing; `'Pacman'` or `'Ghost'` after termination.

Always use the **state** for current players and food; `grid` is the original layout and may show a dot that has been eaten or an old ghost position. A1's tuples, lists, dictionaries and frozensets are used again; the new state fields record moving ghosts, remaining food, score and the winner. Use the supplied `support.constants` for A1's symbols and direction names. The agent label `'P'` identifies Pacman; it is distinct from the board symbol `●`. Ghost label `'W'` is distinct from the direction `'West'`. Ghost labels in `problem` and `state` match, but may skip labels: `W` and `Y` need not include `X`. Maps have one to four ghosts. Inputs are valid states; search depth is an integer. You need not implement JSON parsing.

Do not modify any input problem or state. Q1 successors must create a new state dictionary and a new inner `ghosts` dictionary, so changing one branch cannot affect its parent or siblings. Tuples and frozensets are

immutable and may be shared; create a new frozenset when food changes. Merely copying the outer dictionary leaves the inner ghost dictionary shared.

Q0: Manhattan Distance (0 Marks)

Implement `manhattan(a, b)` in `q0.py`: return the integer Manhattan distance between two coordinate tuples. Add the absolute row difference and absolute column difference. Ignore walls. Do not change either input.

In the corridor, `(5, 1)` and `(4, 1)` are one row apart. The Q0 case supplies these coordinates; it does not ask for a route to a food dot. Coordinates may be negative, and identical positions have distance zero.

```
python3 q0.py corridor
python3 q0.py --test
```

The first prints `Distance: 1`; the second should pass **3/3** checks. You may reuse `manhattan` in Q2/Q4. Q0 is not submitted; marking supplies a correct version of it. Complete it locally so your later code can run.

Q1: Game Rules and Random Play (20 Code Marks)

1. Implement Legal Moves, State Updates and Random Choices

In the corridor, Pacman starts at `(5, 1)`. Its legal moves are `North`, then `East`. East eats a dot and leaves one dot: score `0 - 1 + 10 = 9`. North enters an empty cell: score `-1`. Neither move ends the game. Use these two examples to check your state updates before running a whole game.

Complete the three functions in `q1.py`, following the preceding rules:

- **`legal_actions(problem, state, agent)`** returns a list of legal direction strings in **North, East, South, West** order. `agent` is `'P'` or a present ghost label. Return `[]` for a terminal state or no legal directions. Ghost blocking applies only to other ghosts; capture moves are still legal.
- **`successor(problem, state, agent, action)`** returns the new state after one action. `action` is a direction or `None` for a forced pass. Return the full state with updated position, food, score and winner. Preserve the parent and problem, including the parent's ghost dictionary. Handle terminal input and reject invalid moves as specified above.
- **`random_action(problem, state, agent, rng)`** returns one direction chosen uniformly using **`rng.choice(legal_actions(...))`**, or `None` if the list is empty. `rng` is a supplied `random.Random` instance. Call its `choice` exactly once when actions exist, zero times for a pass. Do not seed it yourself or use global randomness; the runner manages reproducibility.

2. Test Each Function

Implement and check one function at a time:

```
python3 q1.py --test --stage actions
python3 q1.py corridor --inspect
python3 q1.py --test --stage successor
python3 q1.py east_food --inspect
python3 q1.py --test --stage random
python3 q1.py --test
```

The checkpoints are **7** legality checks, **13** successor/invalid-action checks and **2** randomness checks. The full suite includes a seeded game: aim for **23/23**. An unfinished function fails its checks without skipping later ones. Tests include open borders, food, capture on food, ghost blocking, passes, terminal copies, input types and RNG use. Q1 supports multiple ghosts too. The checks compare function results, not printed formatting.

3. Run and Replay a Game

Q1 gameplay makes Pacman and each present ghost choose randomly. After completing Q1, run this command (result excerpt):

```
$ python3 q1.py corridor --seed 8 --max-rounds 40
Trial 1: Ghost, score -494, Pacman turns 4
Wins: 0/1 | Losses: 1 | Unfinished: 0
```

`--max-rounds 40` allows up to 40 rounds, each consisting of Pacman's turn followed by every ghost's turn. The game stops earlier if someone wins; otherwise, it reports **Unfinished** after 40 rounds.

```
python3 q1.py corridor --seed 8 --max-rounds 40 --replay
python3 play.py corridor --seed 8
```

Replay runs your game first, then shows recorded moves. Press Enter for the next frame or `q` to stop playback. Manual play requires completed Q1. As in A1, use arrow keys or W/A/S/D, `R` to restart and `Q` to quit. When input is redirected, use W/A/S/D followed by Enter. Its ghosts use your random function. `python3 play.py` uses the larger `crossroads` map. No additional packages are needed. Submit `q1.py`.

Q2: Reflex Pacman with One Ghost (10 Code + 5 Written Marks)

A reflex player evaluates **one Pacman move** at a time, without building a multi-turn tree. The ghost still chooses randomly. Complete Q0 and Q1 first.

1. Write an Evaluation Function

```
python3 q2.py race --show-maze
```

Implement **`evaluate_action(problem, state, action)`** in `q2.py`. Apply your Q1 successor to obtain the state after Pacman's action, then return a **finite integer or float**: higher means better for Pacman. Do not change inputs. `action=None` represents a forced pass.

For a terminal successor, return its **final game score exactly**. For other successors, design a formula using at least the nearest food and nearest ghost Manhattan distances. You choose the weights and may use score or other sensible features. Explain their signs and relative strength in your written answer. Prefer an immediate win and avoid immediate capture when a safe move exists. Handle any empty-food input safely; never divide by zero.

Your nonterminal formula must use both required distances meaningfully. A formula using only score and food count does not meet Q2's feature requirements, even if it passes the basic contract checks.

The supplied runner evaluates all legal Pacman actions, or the one forced pass, and selects the **first maximum** in North/East/South/West order. You implement the evaluation, not this action-selection loop. Do not search multiple turns in Q2.

```
python3 q2.py corridor --inspect
python3 q2.py corridor
python3 q2.py --test
python3 q2.py single_crossroads --replay
```

Aim for **6/6** public checks. These check finite numbers, input preservation, terminal scores, an immediate win and avoiding immediate capture. They do not require your chosen nonterminal values to match a reference formula. Evaluation quality and explanation are marked separately.

2. Compare with Random Play

Use identical maps and trial settings, after Q1/Q2 are complete:

```
python3 q1.py corridor --trials 10 --seed 8 --max-rounds 40
python3 q2.py corridor --trials 10 --seed 8 --max-rounds 40
python3 q2.py race --trials 10 --seed 8 --max-rounds 40
```

Trial 1 uses seed 8, trial 2 seed 9, through seed 17. Rerunning unchanged code and settings reproduces results. Different policies may consume random choices at different times; equal seeds do not mean identical ghost paths. A seed makes an experiment reproducible, not proof that the policy generalises.

In `q2_answers.md`, report wins, losses and unfinished trials for all three runs. In **at most 150 words**, describe your formula, explain one observed difference from random play, and identify a limitation of one-step reasoning. Report your own measurements even if your player loses. Submit `q2.py` and `q2_answers.md`.

Q3: Multiple Agents and Turn Depth (3 Code + 2 Written Marks)

Reuse Q1's game rules. Q3 gameplay makes **every agent random**, now allowing one to four ghosts. Complete two small helpers in `q3.py`; later searches reuse these helpers to keep their turn/depth accounting consistent.

1. Implement the Agent Order

`turn_order(problem)` returns a tuple beginning with `'P'`, followed by **only the present ghost labels, alphabetically**. Examples:

```
Ghosts W, X:    ('P', 'W', 'X')
Ghosts W, Y:    ('P', 'W', 'Y')
Ghosts WXYZ:    ('P', 'W', 'X', 'Y', 'Z')
```

Use labels from `problem['ghosts']`, not their positions. Do not modify it.

2. Count Complete Rounds

`next_turn(problem, agent, depth)` returns `(next_agent, next_depth)`. The current agent is present and `depth` is positive. Keep depth unchanged until the **last present ghost** has taken its turn; then return `('P', depth - 1)`. A forced pass still advances the turn.

With P/W/X and depth 2, the calls advance as follows:

```
(P, 2) -> (W, 2) -> (X, 2) -> (P, 1)
       -> (W, 1) -> (X, 1) -> (P, 0)
```

This assignment's depth is **complete rounds**, unlike a lecture tree whose levels count individual moves. At a Pacman root, depth 2 permits two Pacman turns and two turns for each ghost, unless the game ends sooner. At a mid-round ghost root, count the rest of that round, then subsequent complete rounds: starting at `(W, 2)` above reaches `(P, 0)` after W, X, P, W, X. Stop search before expanding any node at depth zero.

3. Test Turn Order and Depth

```
python3 q3.py two_ghosts --inspect
python3 q3.py four_ghosts_last_ghost --inspect
python3 q3.py written_turns --inspect
python3 q3.py --test
python3 q3.py crossroads --seed 8 --max-rounds 40 --replay
```

`--inspect` prints the helper result for the case's current agent and depth, without playing a game. Aim for **10/10** checks. Then observe a multi-ghost game with replay: each ghost moves after the previous one's new position is known, so legal moves can change during a round.

In `q3_answers.md`, use the ghost labels from `written_turns`. On this map, Z is trapped and must pass. Assume the game remains nonterminal during this turn-accounting exercise. A faulty implementation gives this trace from `(X, 2)`:

```
(X, 2) -> (Z, 1) -> (P, 0)
```

Write the correct sequence through the first depth-zero turn. In **at most 80 words**, identify the first incorrect transition, explain its effect on the number of Pacman turns considered, and show how Z's forced pass appears in your corrected sequence. Submit `q3.py` and `q3_answers.md`.

Q4: Reflex Pacman with Multiple Ghosts (7 Code + 3 Written Marks)

Keep the same game rules. Pacman still evaluates a single move; every ghost chooses randomly. Complete Q1 and your Q0 distance helper first. Gameplay and replay also require Q3's `turn_order`; Q4 tests and inspection do not.

1. Examine a Map with Two Ghosts

Preview the map (complete output):

```
$ python3 q4.py two_ghosts --show-maze
Running Q4: two_ghosts
Map: maps/two_ghosts.json
  0123
0 ████
1 █·X█
2 █W ▌
3 ▌● ▌
4 ████
Score: 0 | Food remaining: 1
```

Look at the positions of both ghosts. In Q4, your evaluation must consider all ghosts, rather than just the single ghost used in Q2.

2. Extend Your Evaluation to Multiple Ghosts

Implement **`evaluate_action(problem, state, action)`** in `q4.py`. As in Q2, return a finite number, with higher values indicating better moves for Pacman. Return the exact final score for a terminal successor, handle forced passes and leave the inputs unchanged. Prefer immediate wins and avoid immediate capture when a safe move exists. For nonterminal successors, consider food and **all present ghosts**, and include **one additional risk or escape feature**. For example, you could consider nearby-ghost counts or legal exits. Explain what your feature measures; you choose its formula and weights. You may reuse your Q2 reasoning or small helpers. Do not duplicate the game engine or implement a multi-turn search here.

Also implement **`evaluate_action_without_feature(problem, state, action)`** in the same file. Return the same evaluation with **only your additional risk/escape feature removed**. Keep every other feature, weight and terminal score unchanged. Both functions must return finite numbers, leave inputs unchanged and handle forced passes. They may share helpers. Comparing them lets you see what your additional feature changes.

```
python3 q4.py --test
python3 q4.py crossroads --replay
python3 q4.py four_ghosts --show-maze
```

Aim for **7/7** checks. Each case checks both evaluation functions for finite values, unchanged inputs and exact terminal scores, including forced passes. Immediate win/capture choices are checked for the full evaluator. These checks do not establish feature quality or whether only the added feature was removed.

3. Compare Moves With and Without Your Added Feature

```
python3 q4.py written_feature --show-maze
python3 q4.py written_feature --inspect --ablate
```

The larger board has several escape routes and three ghosts. The second command prints your full and without-feature values for each legal move, followed by both selected actions. Record these in `q4_answers.md`. In **at most 120 words**, explain your food/all-ghost features, what the additional feature measures and how removing it changes the values or ranking. An unchanged selected move is valid: explain why the feature did or did not alter the decision. These are measurements of your formula, not prescribed reference values. Submit `q4.py` and `q4_answers.md`.

Q5: Minimax (20 Code + 5 Written Marks)

In minimax, **Pacman maximises and every ghost minimises the same Pacman utility.** This models coordinated adversarial ghosts, unlike Q4's random ghosts. Q5 gameplay calls minimax for Pacman **and each ghost** at its actual turn. Complete Q1 and Q3 first; Q2/Q4 evaluations are not used.

1. Implement the Minimax Function

```
python3 q5.py corridor --show-maze
```

Implement **`minimax(problem, state, agent='P', depth=2)`** in `q5.py`. Keep these arguments and defaults. `agent` is the player to move; `depth` is a nonnegative number of remaining round completions, using Q3's rule. Return a dictionary with **exactly**:

- `'action'`: the best legal direction, or `None` for a forced pass or leaf. At a ghost root, this is the ghost's **minimising** action.
- `'value'`: the finite backed-up numeric utility, always from Pacman's viewpoint.
- `'expanded'`: the nonnegative integer number of nodes expanded in this call's complete search tree, including the root if it is expanded.

Use the imported `legal_actions`, `successor`, `next_turn` and supplied `evaluate`. Do not modify inputs or use random choices.

2. Use the Fixed Leaf Evaluator

For terminal states or depth zero, return `action=None`, `expanded=0` and the supplied **`evaluate(problem, state)`** value. Terminal utility is the final score, without another bonus or penalty. For a nonterminal state the evaluator is:

```
score - 10 * number_of_remaining_food
```

This formula is already implemented in `support/`; use it unchanged in both Q5/Q6. It measures score and food remaining without distance features. Search must discover future food collection and captures through

state transitions. Use the same evaluator for both algorithms so their backup rules can be compared. Keep your independently designed Q2/Q4 reflex formulas separate.

3. Evaluate Every Child and Count Expanded Nodes

For a nonterminal node with positive depth, obtain legal actions in Q1's order. If none exist, examine **one `None` successor** for the forced pass. Generate each child with Q1 and use Q3 to find its next agent and depth. Recurse on **every** child. A Pacman node selects the maximum value; a ghost node selects the minimum. On equal values, retain the **first** action in North/East/South/West order.

Count this expanded node as 1 and add all children's expansion counts. Terminal and cutoff nodes count zero. Count tree occurrences, even when different paths reach an equal state. Do **not** use an explored set, memoisation or alpha-beta pruning in this assignment; those change the required full-tree count.

4. Test Your Search and Run a Game

`--inspect` calls your search once on the case's state and root agent. It prints Action, Value and Expanded nodes, without playing a game or reading expected answers. Most cases use the initial board; cases with a supplied different state show that state separately. `--depth` overrides the case's depth for inspection.

```
python3 q5.py corridor --inspect --depth 1
python3 q5.py corridor --inspect --depth 2
python3 q5.py --test
```

Aim for **21/21**. Tests include terminal and depth-zero leaves, ties, passes, ghost roots at different points within a round and multiple ghosts. Normal gameplay requires positive depth; `--inspect --depth 0` is allowed to inspect a cutoff leaf.

```
python3 q5.py escape --depth 2 --max-rounds 20 --replay
```

The game runner starts a **new search** at each actual agent's turn, using the configured depth and Q3's rule for counting rounds from a ghost turn. At a Pacman root, depth 2 sees two full rounds; at a W root it sees the rest of the current round and the next round.

Game output's `expanded` is the **sum across all search calls in that trial**. Replay labels this as the count across the completed game, not the count of nodes drawn on the board. `--inspect` and test output instead count **one search call**. Q1-Q4 do not display this count because they do not build search trees. Increasing depth or adding ghosts can make runs much slower. Start with small maps and depth 1 before using depth 2; no higher depth is required.

5. Compare Depths 1 and 2

```
python3 q5.py written_shallow --inspect
python3 q5.py written_deep --inspect
```

These cases use the **same map and initial state**, with depths 1 and 2. Record action, value and expanded count for each. Then inspect an actual ghost backup:

```
python3 q5.py written_backup --inspect --branches
```

Record each root action and child value. Calculate the **minimum** and identify which action the root should return, using the first-action tie rule. The branch values come from additional calls to your code; their diagnostics do not add to the root expansion count. In **at most 150 words**, explain this backup, the extra turns at depth 2 and why a deeper finite search still cannot guarantee a win. If your horizon comparison retains the same action, explain the changed values and horizon. Submit `q5.py` and `q5_answers.md`.

Q6: Expectimax (16 Code + 9 Written Marks)

Model the ghosts as **random**, as in Q4. Pacman still selects its best action, but ghost nodes now average possible outcomes. Reuse Q1/Q3 and Q5's leaf, return and expansion-count contracts. Keep the same supplied evaluator.

1. Use Chance Nodes for Ghost Turns

Implement `expectimax(problem, state, agent='P', depth=2)` in `q6.py`. At terminal or depth-zero nodes, return the supplied evaluation, action `None` and expansion count zero. At Pacman nodes, examine every child and choose the first maximum in North/East/South/West order, exactly as in Q5.

At a ghost node, examine **all currently legal actions** and use their **arithmetic mean** as the node's value. Each has probability `1 / number_of_actions`. Recompute this legal list at each node: earlier ghosts can change which cells are occupied. A forced pass has one child with probability 1. Ghosts do not optimise the utility. Do not sample random actions inside the search.

Return `'action': None` at a ghost-root chance node: this describes a distribution, not a chosen direction. Its `'value'` is the expectation and its `'expanded'` counts this node plus every child's expanded count. At a Pacman root return the chosen action, as before. Use full-round depth, no input mutation and no pruning or memoisation. Floating-point means are accepted within a small numerical tolerance.

For example, if a ghost's three legal children have values `-6`, `3` and `9`, the chance node value is `2`; a MIN node would instead use `-6`. These are illustrative values, not an answer to any supplied case.

2. Test Expectimax and Compare It with Minimax

```
python3 q6.py corridor --inspect --depth 1
python3 q6.py --test
python3 q5.py written_two_ghosts --inspect --depth 2
python3 q6.py written_two_ghosts --inspect --depth 2
python3 q5.py written_trap --inspect --depth 2
python3 q6.py written_trap --inspect --depth 2
```

Aim for **21/21**. The tests cover uniform chance values, ties, passes, ghost roots, depth and expansion counts. Record the comparisons before interpreting the results. Compare one search call on identical inputs, not totals from games whose subsequent played states and opponents may differ.

3. Run a Game with Random Ghosts

```
python3 q6.py escape --depth 2 --seed 8 --max-rounds 20 --replay
```

In Q6 gameplay, only **Pacman** calls expectimax. Actual ghosts use Q1's seeded random policy, matching the chance-node model. Q5 gameplay uses minimax ghosts; therefore comparing their win rates also changes the actual opponents. Use the one-call `--inspect` comparisons above to isolate the backup-rule difference.

4. Calculate the Value at a Ghost Node

```
python3 q5.py written_backup --inspect --branches
python3 q6.py written_backup --inspect --branches
```

In `q6_answers.md`, record the four Q5/Q6 comparison rows above. For this additional ghost-root case, record each action's Q5 child value, Q6 child value, Q6 child expansion count and uniform probability. These are outputs of your own code.

1. Calculate the uniform expectation from the Q6 child values. Compare it with the reported root value.
2. Identify the action with the lowest Q5 child value. Use specific numbers from both runs to explain how the two models treat this risky branch. Do not reuse Q5 child values in the Q6 calculation: they use a different ghost model.
3. Change **only this root ghost's** policy: give the first legal action in the stated direction order probability `1/2`; share the other `1/2` equally among the remaining legal actions. Calculate the new root expectation from the same Q6 child values. Descendant ghosts remain uniform; do not change your code.
4. Reconstruct the Q6 root expansion count from the child counts and compare it with the reported count. Explain whether changing only these probabilities requires visiting a different set of nodes under the required full-tree search.

Keep your explanations within **220 words**; tables and arithmetic are excluded. Use the branch values you recorded and discuss an action or value difference from the earlier comparisons. Submit `q6.py` and `q6_answers.md`.

Finish and Submit

Open `q2_answers.md`-`q6_answers.md` in your editor, replace the blank responses with your observations and explanations, and save them with their existing names. Markdown is plain text; no special software is needed. Word limits apply to your explanatory prose, not the supplied prompts, measurement tables or arithmetic.

Check the Saved Files

Run from your extracted `student/` folder:

```
python3 q0.py --test
python3 q1.py --test
python3 q2.py --test
python3 q3.py --test
python3 q4.py --test
python3 q5.py --test
python3 q6.py --test
```

The public checkpoints are **3, 23, 6, 10, 7, 21 and 21** respectively. Check the stated function requirements as well as the test results. Tests may require an earlier dependency: Q2/Q4 use Q0/Q1, Q3 gameplay uses Q1, and Q5/Q6 use Q1/Q3. The Q3 helper tests can pass before Q1 is complete. Q5 does not depend on your Q4 evaluator, and Q6 does not depend on your Q5 implementation. Keep the supplied imports and runners; you may add standard-library imports and your own small helpers.

Upload Eleven Individual Files

Submit these files to **Assignment 2 on the COMP3270 Moodle course page**:

- `q1.py`, `q2.py`, `q3.py`, `q4.py`, `q5.py`, `q6.py`
- `q2_answers.md`, `q3_answers.md`, `q4_answers.md`, `q5_answers.md`, `q6_answers.md`

Upload the files **individually, without a ZIP archive**. Keep the filenames unchanged. Do not submit Q0, `play.py`, maps, support files, test cases, expected answers, the PDF or caches. If you create helpers for your implementation, keep them inside the relevant question files so the eleven submitted files are self-contained alongside the supplied framework.

Check that all eleven files appear in your Moodle submission and that the uploaded versions contain your latest saved work. Submit by **8 November 2026**.